

St. Francis Institute of Technology, Mumbai-400 103
Department of Information Technology

A.Y. 2025-2026

Class: BE Honours Course, Semester: VII

Subject: **AI-ML in Healthcare Lab**

Experiment – 1: To collect, clean, integrate and transform Healthcare data based on specific disease.

- 1. Aim:** To collect, clean, integrate and transform Dataset.
- 2. Objectives:** After study of this experiment, the student will be able to
 - Understand pre-processing concepts
 - Understand NumPy, Panda concepts
- 3. Outcomes:** After study of this experiment, the student will be able to
 - Understand data preparation, NumPy and Pandas.
- 4. Prerequisite:** Fundamentals of Python Programming and Database Management System.
- 5. Requirements:** Python Installation, Personal Computer, Windows operating system, Internet Connection, Microsoft Word.
- 6. Pre-Experiment Exercise:**

Brief Theory:

Basic Concepts of Reading a dataset, Handling the details of dataset, Handling missing values of the dataset.

7. Laboratory Exercise

A. Procedure:

Apply Preprocessing on Diabetes dataset.

8. Post-Experiments Exercise

A. Extended Theory:

Mention different ways to handle missing values.

Mention the difference between Numpy and Pandas.

B. Questions:

1. Execute any 10 properties and methods of the DataFrame object.

C. Conclusion:

Write the significance of the topic studied in the experiment.

D. References:

1. <https://www.w3schools.com/python/pandas>
2. <https://www.geeksforgeeks.org/difference-between-pandas-vs-numpy/>
3. <https://github.com/ashishpatel26/Pima-Indians-Diabetes-Dataset-Missing-Value-Imputation/blob/master/Readme.md>

```

import pandas as pd
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import KFold, cross_val_score
from sklearn.impute import SimpleImputer

def model_fit(dataset):
    values = dataset.values
    X = values[:, 1:8]
    Y = values[:, 8]

    # Handle missing values if needed
    imputer = SimpleImputer(strategy='mean')
    X = imputer.fit_transform(X)

    lda = LinearDiscriminantAnalysis()
    kfold = KFold(n_splits=3, random_state=7, shuffle=True)
    result = cross_val_score(lda, X, Y, cv=kfold, scoring="accuracy")
    print("Result of LDA:", result.mean())

```

Defined a function `model_fit` to train and evaluate a Linear Discriminant Analysis model using cross-validation.

```

pima = pd.read_csv("pima-indians-diabetes.data.csv", header=None)
print(pima.shape)
print((pima[[1,2,3,4,5]] == 0).sum())
pima.describe()

```

(768, 9)

1 5
2 35
3 227
4 374
5 11
dtype: int64

	0	1	2	3	4	5	6	7	8
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Loads the Pima Indians Diabetes dataset and performs initial data inspection.

```

from IPython.core.display import HTML

HTML("""
<style>
.dataframe tbody tr th {
    vertical-align: top;
}
.dataframe thead th {
    text-align: right;
}
</style>
""")
pima.head()

```

(768, 9)

	0	1	2	3	4	5	6	7	8
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

- Sets display options for pandas DataFrames and shows the first few rows of the dataset.

```

pima[[1, 2, 3, 4, 5]] = pima[[1, 2, 3, 4, 5]].replace(0, np.nan)
print(pima.isnull().sum())

```

0	0
1	5
2	35
3	227
4	374
5	11
6	0
7	0
8	0
dtype: int64	

```

model_fit(pima)

```

Result of LDA: 0.7682291666666666

```

pima.shape

```

(768, 9)

Replaces zeros with NaN in specific columns and checks for missing values. Calls the `model_fit` function with the dataset containing NaNs and prints the shape of the DataFrame after replacing zeros with NaN.

```

import numpy
pima[[1,2,3,4,5]] = pima[[1,2,3,4,5]].replace(0, numpy.nan)
pima.dropna(inplace=True)
pima.shape

```

(392, 9)

```

model_fit(pima)

```

Result of LDA: 0.7935016637306713

```

pima.shape

```

(392, 9)

```

pima[[1,2,3,4,5]] = pima[[1,2,3,4,5]].replace(0, numpy.nan)
# fill missing values with mean column values
pima.fillna(pima.mean(), inplace=True)
pima.shape

```

(392, 9)

Replaces zeros with NaN and then removes rows with missing values, printing the new shape. Calls the `model_fit` function with the dataset after dropping rows with NaNs. Prints the shape of the DataFrame after dropping rows with NaNs and replaces zeros with NaN and then fills missing values with the mean of each column, printing the new shape.

`pima.head(10)`

	0	1	2	3	4	5	6	7	8
3	1	89.0	66.0	23.0	94.0	28.1	0.167	21	0
4	0	137.0	40.0	35.0	168.0	43.1	2.288	33	1
6	3	78.0	50.0	32.0	88.0	31.0	0.248	26	1
8	2	197.0	70.0	45.0	543.0	30.5	0.158	53	1
13	1	189.0	60.0	23.0	846.0	30.1	0.398	59	1
14	5	166.0	72.0	19.0	175.0	25.8	0.587	51	1
16	0	118.0	84.0	47.0	230.0	45.8	0.551	31	1
18	1	103.0	30.0	38.0	83.0	43.3	0.183	33	0
19	1	115.0	70.0	30.0	96.0	34.6	0.529	32	1
20	3	126.0	88.0	41.0	235.0	39.3	0.704	27	0

Next steps: [Generate code with pima](#) [New interactive sheet](#)

`model_fit(pima)`

Result of LDA: 0.7935016637306713

Displays the first 10 rows of the DataFrame after filling missing values with the mean and calls the `model_fit` function with the dataset after filling missing values with the mean.

```
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import KFold, cross_val_score

# Replace zeros with NaN in selected columns
pima[[1, 2, 3, 4, 5]] = pima[[1, 2, 3, 4, 5]].replace(0, np.nan)

# Separate features and labels
values = pima.values
X = values[:, 1:8]
Y = values[:, 8]

# Impute missing values (mean strategy)
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)

# Confirm no missing values remain
print("Missing values after imputation:", np.isnan(X_imputed).sum())

# Run LDA with cross-validation
lda = LinearDiscriminantAnalysis()
kfold = KFold(n_splits=3, random_state=7, shuffle=True)
result = cross_val_score(lda, X_imputed, Y, cv=kfold, scoring='accuracy')

# Print the result
print("Result of LDA (mean accuracy):", result.mean())
X_imputed_df = pd.DataFrame(X_imputed)
print(X_imputed_df.isnull().sum())
```

Missing values after imputation: 0
 Result of LDA (mean accuracy): 0.7935016637306713
 0 0
 1 0
 2 0
 3 0
 4 0
 5 0
 6 0
 dtype: int64

Performs data imputation using `SimpleImputer` and runs LDA with cross-validation, confirming no missing values.

POST EXPERIMENT EXERCISE:

B) Execute any 10 properties and methods of the DataFrame object.

```

import pandas as pd
import numpy as np

# Load the dataset
pima = pd.read_csv("pima-indians-diabetes.data.csv", header=None)

# Replace zeros with NaN in selected columns
pima[[1, 2, 3, 4, 5]] = pima[[1, 2, 3, 4, 5]].replace(0, np.nan)

# Fill missing values with the mean of each column
pima.fillna(pima.mean(), inplace=True)

# Execute and display 10 properties and methods of the DataFrame object
print("Shape:", pima.shape)
print("\nInfo:")
pima.info()
print("\nDescribe:")
display(pima.describe())
print("\nColumns:", pima.columns)
print("\nIndex:", pima.index)
print("\nData types:\n", pima.dtypes)
print("\nNumber of non-null values per column:\n", pima.count())
print("\nMean of each column:\n", pima.mean())
print("\nCorrelation matrix:\n", pima.corr())
print("\nFirst 5 rows:\n")
display(pima.head())

```

Shape: (768, 9)

Info:
 <class 'pandas.core.frame.DataFrame'>
 RangeIndex: 768 entries, 0 to 767
 Data columns (total 9 columns):
 # Column Non-Null Count Dtype
 --- ---
 0 0 768 non-null int64
 1 1 768 non-null float64
 2 2 768 non-null float64
 3 3 768 non-null float64
 4 4 768 non-null float64
 5 5 768 non-null float64
 6 6 768 non-null float64
 7 7 768 non-null int64
 8 8 768 non-null int64
 dtypes: float64(6), int64(3)
 memory usage: 54.1 KB

Describe:

	0	1	2	3	4	5	6	7	8
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.686763	72.405184	29.153420	155.548223	32.457464	0.471876	33.240885	0.348958
std	3.369578	30.435949	12.096346	8.790942	85.021108	6.875151	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	25.000000	121.500000	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.202592	29.153420	155.548223	32.400000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	155.548223	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Columns: Index([0, 1, 2, 3, 4, 5, 6, 7, 8], dtype='int64')

Index: RangeIndex(start=0, stop=768, step=1)

Data types:

```

0    int64
1    float64
2    float64
3    float64
4    float64
5    float64
6    float64
7    int64
8    int64
dtype: object

```

Number of non-null values per column:

```

0    768
1    768
2    768
3    768
4    768
5    768
6    768
7    768
8    768
dtype: int64

```

NAME: Tanmay Bhatkar Class: BE INFT A Sr.No/Roll no:8

Mean of each column:

```
0      3.845052
1    121.686763
2     72.405184
3     29.153420
4    155.548223
5     32.457464
6      0.471876
7     33.240885
8      0.348958
dtype: float64
```

Correlation matrix:

```
      0      1      2      3      4      5      6
0  1.000000  0.127911  0.208522  0.082989  0.056027  0.021565 -0.033523
1  0.127911  1.000000  0.218367  0.192991  0.420157  0.230941  0.137060
2  0.208522  0.218367  1.000000  0.192816  0.072517  0.281268 -0.002763
3  0.082989  0.192991  0.192816  1.000000  0.158139  0.542398  0.100966
4  0.056027  0.420157  0.072517  0.158139  1.000000  0.166586  0.098634
5  0.021565  0.230941  0.281268  0.542398  0.166586  1.000000  0.153400
6 -0.033523  0.137060 -0.002763  0.100966  0.098634  0.153400  1.000000
7  0.544341  0.266534  0.324595  0.127872  0.136734  0.025519  0.033561
8  0.221898  0.492928  0.166074  0.215299  0.214411  0.311924  0.173844

      7      8
0  0.544341  0.221898
1  0.266534  0.492928
2  0.324595  0.166074
3  0.127872  0.215299
4  0.136734  0.214411
5  0.025519  0.311924
6  0.033561  0.173844
7  1.000000  0.238356
8  0.238356  1.000000
```

First 5 rows:

	0	1	2	3	4	5	6	7	8
0	6	148.0	72.0	35.00000	155.548223	33.6	0.627	50	1
1	1	85.0	66.0	29.00000	155.548223	26.6	0.351	31	0
2	8	183.0	64.0	29.15342	155.548223	23.3	0.672	32	1
3	1	89.0	66.0	23.00000	94.000000	28.1	0.167	21	0
4	0	137.0	40.0	35.00000	168.000000	43.1	2.288	33	1